

QP-3.1 - Linguagens de Programação: Questões pré-classe para a aula 1 de "Nomes, Vinculações, Verificações de Tipo e Escopo"

Total de pontos 23/23 ?

E-mail *

fgiovanni@discente.ufg.br

0 de 0 pontos

Qual é o seu nome? (para registro das respostas) *

GIOVANNI DE OLIVEIRA FELISBERTO

2 de 2 pontos

✓ Qual o significado de valor_r de uma variável? *

1/1

- ☒ É o conteúdo armazenado na posição de memória à qual a variável está associada. ✓
- ☐ É o endereço de memória ao qual a variável está associada.
- ☐ É o conteúdo armazenado no nome da variável.
- ☐ É o endereço de memória onde está armazenado o nome (identificador) da variável.



✓ Qual o significado de valor_l de uma variável? *

1/1

- ☐ É o conteúdo armazenado no nome da variável.
- ☒ É o endereço de memória ao qual a variável está associada. ✓
- ☐ É o conteúdo armazenado na posição de memória à qual a variável está associada.
- ☐ É o endereço de memória onde está armazenado o nome (identificador) da variável.

1 de 1 pontos

Considere o seguinte código C:

```
int a = 0;  
int b = a;
```

✓ No código C anterior, é possível dizer: *

1/1

- ☒ As variáveis possuem o mesmo valor_r, mas valor_l diferente. ✓
- ☐ As variáveis possuem valor_r e valor_l diferentes.
- ☐ As variáveis possuem o mesmo valor_l, mas valor_r diferentes.
- ☐ As variáveis possuem o mesmo valor_l e valor_r.

1 de 1 pontos

Considere as seguintes afirmativas:

1. As duas variáveis podã armazenar a mesma faixa de valores.
2. As duas variáveis terão o mesmo valor_r.
3. As duas variáveis terão o mesmo valor_l.
4. As duas variáveis ocuparão mesmo número de bytes na memória.



✓ Quais das afirmativas estão corretas? *

1/1

- ☐ 2 e 4
- ☐ 1, 3 e 4
- ☐ todas as afirmativas estão corretas.
- ☐ 1, 2 e 3
- ☐ 1
- ☐ 3 e 4
- ☐ 1, 2 e 4
- ☐ 1 e 2
- ☐ 2, 3 e 4
- ☒ 1 e 4
- ☐ 4



19 de 19 pontos

Considere o seguinte trecho de código C e o seguinte mapa em memória das variáveis `v` e `a`. Considere também que o tipo `int` utiliza 4 bytes, e que o tipo `char` utiliza 1 byte. Cada endereço no mapa de memória representa 1 byte (a numeração está saltando de 4 em 4 bytes).

```
int v[] = {1, 2, 3}; // linha 1
char a[] = {'1', '2', '3', '4'}; // linha 2
a[1] = v[1]; // linha 3
```

```
+-----+
v |      | 0x0001
+-----+
|      | 0x0005
+-----+
|      | 0x0009
+-----+
a |      | 0x000D
+-----+
```



✓ Qual é o valor_l e o valor_r das variáveis v[1] e a[1], respectivamente, 1/1
ANTES da execução da linha 3? *

- ☐ 0x0005, 2, 0x0010, '2'
- ☐ 0x0005, 1, 0x000E, '1'
- ☐ nenhuma das demais afirmativas está correta.
- ☐ 0x0001, 2, 0x000E, '2'
- ☒ 0x0005, 2, 0x000E, '2' ✓
- ☐ 0x0001, 2, 0x000D, '2'
- ☐ 0x0005, 2, 0x000D, '2'

✓ Para a mesma questão anterior, qual é o valor_l e o valor_r das variáveis 1/1
v[1] e a[1], respectivamente, DEPOIS da execução da linha 3? (Para
verificar a representação binária de diferentes números, você pode
utilizar o código deixado em <https://pastebin.com/ZywDnSfD>) *

- ☐ 0x0005, 2, 0x000E, 0x32
- ☐ nenhuma das demais afirmativas está correta.
- ☐ 0x0001, 2, 0x0005, 0x00
- ☐ 0x0001, 2, 0x000E, 0x32
- ☐ 0x0005, 2, 0x0005, 0x00
- ☐ 0x0005, 2, 0x0005, 0x32
- ☐ 0x0001, 2, 0x000E, 0x00
- ☒ 0x0005, 2, 0x000E, 0x00 ✓



✓ Uma amarração que é definida independentemente o código escrito ou 1/1
contexto de execução, é realizada *

- ☐ Na compilação do programa
- ☐ Na ligação do programa (link)
- ☐ Na implementação da linguagem
- ☒ Na definição da linguagem ✓
- ☐ Na carga ou execução do programa

✓ Uma amarração que é definida independentemente o código escrito mas 1/1
que pode variar de um sistema operacional para outro, é realizada *

- ☒ Na implementação da linguagem ✓
- ☐ Na compilação do programa
- ☐ Na definição da linguagem
- ☐ Na ligação do programa (link)
- ☐ Na carga ou execução do programa



✓ Uma amarração que é definida quando o endereço dos módulos (blocos 1/1 de código com funções e variáveis, por exemplo) que fazem parte de um programa , é realizada *

- ☐ Na definição da linguagem
- ☐ Na compilação do programa
- ☐ Na implementação da linguagem
- ☒ Na ligação do programa (link) ✓
- ☐ Na carga ou execução do programa

✓ Em C, quando ocorre a amarração sobre o tipo de uma variável? * 1/1

- ☐ Na ligação do programa (link)
- ☐ Na definição da linguagem
- ☒ Na compilação do programa ✓
- ☐ Na implementação da linguagem
- ☐ Na carga ou execução do programa

✓ Em C, quando ocorre a amarração sobre os possíveis valores que um inteiro (`int`) pode assumir? * 1/1

- ☒ Na implementação da linguagem ✓
- ☐ Na ligação do programa (link)
- ☐ Na compilação do programa
- ☐ Na definição da linguagem
- ☐ Na carga ou execução do programa



✓ Em C, quando ocorre a amarração sobre o endereço de uma variável local a uma função? *

1/1

- ☐ Na implementação da linguagem
- ☐ Na carga do programa
- ☐ Na definição da linguagem
- ☐ Na ligação do programa (link)
- ☒ Na execução do programa
- ☐ Na compilação do programa



✓ Em Java, quando ocorre a amarração do significado da palavra reservada `class`? *

1/1

- ☐ Na ligação do programa (link)
- ☐ Na implementação da linguagem
- ☐ Na compilação do programa
- ☐ Na carga ou execução do programa
- ☒ Na definição da linguagem



✓ Em Java, quando ocorre a amarração sobre os possíveis valores que um inteiro (`int`) pode assumir? * 1/1

- ☐ Na compilação do programa
- ☐ Na implementação da linguagem
- ☐ Na ligação do programa (link)
- ☐ Na carga ou execução do programa
- ☒ Na definição da linguagem



✓ Em Java, quando ocorre a amarração sobre a implementação de uma classe usada em um programa? * 1/1

- ☐ Na implementação da linguagem
- ☐ Na definição da linguagem
- ☒ Na ligação do programa (link)
- ☐ Na compilação do programa
- ☐ Na carga ou execução do programa



✓ A amarração de tipo especifica * 1/1

- ☒ Qual tipo está associado a um nome em um programa.
- ☐ Quanto bytes ocupa um tipo em uma linguagem.
- ☐ Todas as demais respostas estão corretas.
- ☐ Se uma variável (nome) poderá mudar de tipo durante a execução de um programa.



 Na amarração de tipo estática *

1/1

- ☐ O número de bytes ocupado por um tipo é definido na especificação da linguagem e por isso não pode ser mais alterado.
- ☐ Os valores das variáveis não podem ser alterados durante a execução de um programa.
- ☐ O número de bytes ocupado por um tipo é definido antes da execução de um programa e não pode mais ser alterado.
- ☐ Apenas os endereços das variáveis não podem ser alterados durante a execução de um programa.
- ☒ Os tipos das variáveis são amarrados/associados antes da execução de um programa e não podem ser mais alterados. 

 Na amarração de tipo dinâmica *

1/1

- ☐ Os tipos das variáveis são amarrados/associados durante a execução de um programa, mas não podem ser mais alterados uma vez definidos.
- ☐ Os endereços das variáveis podem ser alterados durante a execução de um programa.
- ☒ Os tipos das variáveis são amarrados/associados durante a execução de um programa. 
- ☐ Os tipos das variáveis são amarrados/associados antes da execução de um programa, mas podem ser mais alterados.
- ☐ O número de bytes ocupado por um tipo é definido durante da execução de um programa.



✓ Python é uma linguagem que permite amarração dinâmica de tipos. 1/1
Considere a sentença $A = B + C$. O operador "+" pode ser utilizado tanto para a soma de números para a concatenação de strings. *

- ☐ Se B é real ou inteiro então C é real ou inteiro.
- ☐ Se B é real ou C é real, então A é real.
- ☐ Se B é string, então A é string.
- ☐ B é inteiro e C é inteiro
- ☐ Como Python permite amarração dinâmica, quaisquer combinações de tipo de B e C são permitidas.
- ☐ B é real e C é real ou inteiro.
- ☒ Se B é string, então C é string. Se C é string então B é string. ✓

✓ O que é necessário em uma linguagem de programação para que seja possível amarração dinâmica de tipos? 1/1 *

- ☒ A linguagem deve permitir construir sentenças cujos tipos das variáveis envolvidas só possam ser inferidos em tempo de execução (quando o programa estiver executando). ✓
- ☐ A linguagem deve permitir que o programador especifique variáveis que possam ser armazenadas nas áreas de heap e pilha (explícita ou implicitamente).
- ☐ A linguagem não deve possuir tipos.
- ☐ A linguagem não deve especificar o valor de retorno de funções.
- ☐ A linguagem deve permitir conversão automática de tipos.



✓ A amarração de armazenagem específica *

1/1

- ☐ Se uma linguagem permitirá alocação dinâmica de memória ou não.
- ☐ A área da memória - pilha, heap ou outra - em que uma variável será armazenada.
- ☒ O momento em que o endereço (valor_l) de uma variável será especificado e se ele variará ao longo da execução do código. ✓
- ☐ O momento em que o valor (valor_r) de uma variável será especificado e se ele variará ao longo da execução do código.

✓ O compilador é capaz de determinar o endereço de memória de uma variável e colocar as referencias a ele explicitamente no código gerado, quando a variável *

1/1

- ☐ Está associada a área de memória de dados não-inicializados
- ☒ Está associada a área de memória de dados inicializados e não-inicializados. ✓
- ☐ Está associada a área de memória da pilha.
- ☐ Está associada a área de memória da heap ou na pilha.
- ☐ Está associada a área de memória de dados inicializados.

✓ Quais são as condições necessárias e suficientes para que seja possível amarração estática de armazenagem para uma variável ? *

1/1

- ☐ O nome da variável deve estar associado a um mesmo endereço de memória durante toda a execução do programa.
- ☒ O nome da variável deve estar associado a um mesmo endereço de memória em qualquer execução do programa. ✓
- ☐ Variável não pode mudar de valor.
- ☐ Variável deve ser acessível por todo o código.



Este formulário foi criado em Universidade Federal de Goiás.

Google Formulários

